

# **SIMATS ENGINEERING ASSESSMENT TOOL 2**

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA17

---

## **COURSE OUTCOME COVERED**

**CO1:** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.( BL3)

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 3**

**Duration : 1 Hour  
Total Marks:30**

Annexure A

## **Constraint-Based Problem Solving**

---

### ***Code of Conduct:***

*I Hemalakshmi H-192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

## Annexure A

### **Constraint-Based Problem Solving**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
  - Rationality
  - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

**RUBRICS FOR CALCULATION:**

| Criteria                | Weightage | Level 4<br>(Exemplary)                                                         | Level 3<br>(Proficient)                               | Level 2<br>(Developing)                       | Level 1<br>(Beginning)                   |
|-------------------------|-----------|--------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------|------------------------------------------|
| Problem Understanding   | 25%       | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps           | Partial understanding; misses key constraints | Fails to identify constraints correctly  |
| Constraint Analysis     | 25%       | Analyzes constraints effectively and prioritizes them logically                | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization         | No meaningful analysis or prioritization |
| Solution Development    | 25%       | Develops optimal and feasible solutions satisfying all constraints             | Develops workable solutions with minor limitations    | Solutions partially satisfy constraints       | Solutions do not meet constraints        |
| Application of Concepts | 15%       | Effectively applies appropriate concepts, methods, or tools                    | Applies concepts with minor errors                    | Limited application; requires guidance        | Unable to apply relevant concepts        |
| Justification           | 10%       | Provides strong justification for decisions with logical reasoning             | Provides justification with some clarity              | Weak or unclear justification                 | No justification provided                |

1 Answer:

We model the problem as assigning each doctor (D1, D2, D3) to one of the shifts {Morning, Afternoon, Night} subject to the given constraints:

- $D1 \neq \text{Night}$
- $D3 \neq \text{Morning}$
- Only one doctor per shift (all shifts must be different)
- D2 must work before D3 (Morning < Afternoon < Night)
- Each doctor must have exactly one shift

We apply Backtracking Search, assigning doctors in the order D1, D2, D3.

**Step 1: Assign D1**

Domain(D1) = {Morning, Afternoon, Night}, but D1 cannot work Night. So Domain(D1) = {Morning, Afternoon}.

- Try **D1 = Morning** All constraints satisfied so far.

**Step 2: Assign D2**

Domain(D2) = {Morning, Afternoon, Night}, but must respect “only one doctor per shift”.

- Try **D2 = Morning** Conflict: D1 already has Morning → violates “only one doctor per shift”. Reject D2 = Morning.
- Try **D2 = Afternoon** No conflict with D1 (D1 = Morning, D2 = Afternoon). Constraint “D2 before D3” not yet checked fully (D3 unassigned), so continue.

**Step 3: Assign D3**

Domain(D3) = {Morning, Afternoon, Night}, but D3 cannot work Morning. So Domain(D3) = {Afternoon, Night}. Also, D2 must work before D3.

- Try **D3 = Afternoon** Conflict: D2 already has Afternoon → violates “only one doctor per shift”. Reject D3 = Afternoon.
- Try **D3 = Night** Check constraints:
  - $D3 \neq \text{Morning}$  → satisfied.
  - All shifts different: D1 = Morning, D2 = Afternoon, D3 = Night → satisfied.
  - D2 before D3: Afternoon < Night → satisfied.
  - $D1 \neq \text{Night}$  → satisfied.

All constraints satisfied. No need to backtrack further.

**Final valid schedule:**

- D1 → Morning
- D2 → Afternoon
- D3 → Night

---

2 Answer:

We model the 5×5 grid as a state-space search problem. Each cell  $(i, j)$  is a state. The robot must move from Start state  $S$  to Goal state  $G$ .

**Constraints:**

- Movement allowed: Up, Down, Left, Right (no diagonal moves).
- Cost of each move = 1.
- Robot cannot pass through obstacle cells.
- Manhattan Distance heuristic:  $h(n) = |x_n - x_G| + |y_n - y_G|$ .

**Problem formulation:**

- **States:** All reachable grid cells  $(i, j)$  that are not obstacles.
- **Initial state:** Start cell  $S$ .

- **Goal state:** Goal cell  $G$ .
- **Actions:** From a cell  $(i, j)$ , possible actions are: Up:  $(i-1, j)$ , Down:  $(i+1, j)$ , Left:  $(i, j-1)$ , Right:  $(i, j+1)$ , provided the resulting cell is inside the grid and not an obstacle.
- **Path cost:** Number of moves (each move cost = 1).
- **Heuristic:** Manhattan distance used to estimate distance to goal (for evaluation).

#### **BFS search with queue (step-by-step idea):**

1. Initialize queue with Start node: Queue = [S]. Mark S as visited.
2. **Expand S:** Generate all valid neighbors of S (Up, Down, Left, Right) that are inside the grid and not obstacles. Add them to the back of the queue. Example: Queue = [N1, N2, N3, ...].
3. **Expand next node in queue (FIFO):** Dequeue front node (say N1), generate its valid neighbors, skip already visited or obstacle cells, and enqueue new nodes. Continue this process level by level.
4. At each expansion, we can compute Manhattan distance  $h(n)$  for each node to evaluate how far it is from G, but BFS still expands nodes in order of increasing depth (queue order), ensuring the shortest path in number of steps.
5. When the Goal node G is dequeued and expanded (or recognized as goal when generated), BFS stops. The path from S to G is reconstructed by storing parent pointers for each node when it is generated.

#### **Optimal path and cost:**

- BFS explores all nodes in increasing depth, so the first time G is reached, the path found is the **shortest path in terms of number of moves**.
- Let the optimal path from S to G be:  $S \rightarrow n_1 \rightarrow n_2 \rightarrow \dots \rightarrow G$ .
- If this path has  $k$  moves, then the **optimal cost = k** (since each move costs 1).

This satisfies all constraints: valid moves only, no obstacles crossed, and BFS guarantees the minimum-cost path in this uniform-cost grid.

3 Answer:

#### **a) Constraint Analysis and Impact on Decision-Making**

The robot operates in a **partially observable grid**, meaning it cannot see the entire environment and must make decisions with incomplete information. The constraints affect decision-making as follows:

- **Limited movement (Up, Down, Left, Right):** The robot must plan paths only through adjacent cells, increasing the number of steps needed to reach G.
- **Obstacles:** Blocked cells force the robot to re-plan routes dynamically and avoid unsafe or unreachable areas.
- **Limited sensing (adjacent cells only):** The robot cannot know the full map in advance. It must update its knowledge as it moves, making decisions based on newly discovered information.
- **Movement cost = 1; risky zone cost = 2:** The robot must choose paths that minimize total cost, preferring safe cells even if they require slightly longer routes.
- **Dynamic knowledge update:** The robot must behave as an **online search agent**, continuously revising its plan when new obstacles or risky zones are discovered.

These constraints force the robot to behave adaptively, choosing safe, low-cost paths while learning the environment step-by-step.

## b) Solution Approach Using an Online Uniform Cost Search Agent

A suitable strategy is **Uniform Cost Search (UCS)** combined with **online re-planning**.

**Steps in the solution approach:**

1. **Initialize knowledge base** with only the start cell S and unknown surroundings.
2. **Sense adjacent cells** (up, down, left, right). Mark each as:
  - Free
  - Obstacle
  - Risky zone (extra cost)
3. **Use UCS to plan a path** from current position to G using known information.
  - Priority queue stores nodes ordered by cumulative cost.
  - Cost = 1 for normal cells, 3 for risky cells.
4. **Move to the next cell with lowest cost.**
5. **Sense again** from the new position. If new obstacles or risky zones are discovered:
  - Update the map
  - Re-run UCS from the current position
6. **Repeat sensing + re-planning** until the robot reaches G.

This approach ensures the robot always follows the least-cost path based on current knowledge and adapts when new information appears.

## c) Application of AI Concepts

### Intelligent Agent

The robot acts as an intelligent agent because it:

- Perceives the environment through sensors
- Updates its internal model
- Chooses actions that minimize cost
- Acts autonomously to achieve its goal

### Rationality

The robot is rational because it selects actions that maximize performance:

- Avoids obstacles
- Minimizes movement + risk cost
- Re-plans when new information is discovered
- Always chooses the least-cost path available

### Environment Type

The environment is:

- **Partially observable:** robot sees only adjacent cells
- **Dynamic:** new hazards may appear
- **Stochastic:** risky zones increase cost unpredictably
- **Sequential:** each action affects future decisions
- **Unknown:** full map is not available initially

These characteristics justify the need for online search and continuous sensing.

## d) Justification of the Chosen Approach

The **Online Uniform Cost Search agent** is most suitable because:

- UCS guarantees the **least-cost path** when costs vary (normal vs risky zones).
- Online search allows the robot to operate with **partial knowledge**, updating the map as it moves.
- The robot can **re-plan dynamically** whenever new obstacles or risky zones are discovered.
- UCS handles **variable costs** better than BFS or DFS, which cannot optimize cost.
- The approach ensures **safe navigation**, avoiding risky zones unless necessary.

Thus, UCS with online re-planning satisfies all constraints: safety, cost minimization, partial observability, and dynamic decision-making.